

UNIVERSIDAD DE LOS ANDES
Facultad de Ingeniería y Ciencias Aplicadas
Arquitectura de Computadores 2026-20

Proyecto 2

Juego en assembly sobre Pochoco SoC

Integrantes:
Dariagny Fernández
Daniela Rangel
Javiera Sepúlveda
Profesor:
Jorge Gómez

19 de octubre de 2026

1. Introducción

Los sistemas computacionales modernos se basan en un principio común: un procesador que ejecuta instrucciones almacenadas en memoria y que, mediante direcciones específicas, se comunica con el resto del hardware. En muchos diseños, los periféricos de entrada y salida se integran en el mismo espacio de direcciones que la memoria, técnica conocida como Memory-Mapped I/O (MMIO). Así, el programa puede leer botones o escribir en LED y displays con las mismas instrucciones de carga y almacenamiento (lw y sw), sin necesidad de instrucciones especiales.

Este proyecto busca aplicar esa idea de principio a fin mediante un juego de tiempo de reacción implementado sobre Pochoco SoC, con el procesador Espino y una FPGA Lattice iCE40 HX1K en la Go Board. El jugador debe presionar el botón que corresponde al LED encendido; el sistema mide su tiempo de respuesta en décimas de segundo y calcula el promedio de diez respuestas correctas. Para lograrlo, se extendió el hardware con un contador de ciclos, se programó la lógica del juego en assembly RV32E y se desarrolló un assembler propio en Python que traduce ese programa a código máquina y genera el archivo `game.hex` con que se inicializa la memoria.

De esta forma, el proyecto permite comprender de manera práctica cómo una instrucción escrita en assembly se convierte en una palabra de 32 bits, es ejecutada por el procesador e interactúa finalmente con elementos físicos de la placa. Este informe describe la arquitectura de la solución, el juego en assembly, el assembler y las pruebas realizadas, y cierra con las conclusiones.

2. Descripción General del Proyecto

La lógica principal del juego no se construye como un circuito dedicado para cada etapa, sino como un programa en assembly RV32E que Espino obtiene desde la memoria, decodifica y ejecuta. El procesador se comunica con los botones, los LED y los displays mediante periféricos mapeados en memoria.

La solución se divide en tres partes. El archivo `game.s` contiene las reglas del juego; `assembler.py` traduce sus instrucciones a palabras de 32 bits y genera `game.hex`; y el hardware de Pochoco contiene la CPU, la memoria y los periféricos físicos. Se agregó además un contador de ciclos para que el programa pueda implementar esperas y medir tiempos sin modificar el conjunto de instrucciones de Espino.

El flujo completo se resume en la Figura 1: `game.s` es traducido por `assembler.py` a `game.hex`, cuyo contenido se carga en la memoria de Pochoco SoC para que Espino lo ejecute. Luego, las herramientas Yosys y nextpnr sintetizan, ubican y enrutan el diseño, e icepack genera el bitstream que se programa en la Go Board.

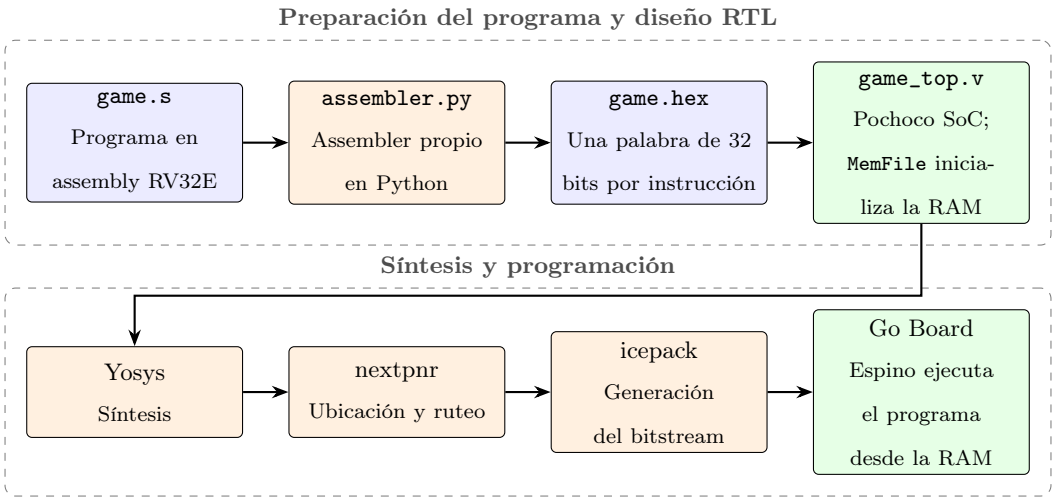


Figura 1: Flujo de trabajo del proyecto, desde `game.s` hasta la ejecución en la Go Board.

Fuente: Elaboración propia.

3. Arquitectura de la Solución

3.1. Procesador, memoria y periféricos

Espino es la CPU de 32 bits que ejecuta el subconjunto RV32E utilizado por el juego. `game.hex` no es un programa distinto de `game.s`, sino su representación en código máquina: cada línea contiene una instrucción de 32 bits escrita con ocho dígitos hexadecimales. La RAM unificada almacena estas palabras y el procesador las obtiene secuencialmente.

Los periféricos se controlan mediante direcciones de memoria. Por ello, `lw` permite leer botones o el contador, mientras que `sw` permite modificar LED y displays. El mapa utilizado es:

Dirección	Periférico
`0x80000000`	Displays de siete segmentos
`0x80000004`	LED
`0x80000008`	Botones
`0x8000000C`	Contador de ciclos

El módulo superior `game_top.v` conserva los puertos físicos de la Go Board, instancia Pochoco SoC y selecciona explícitamente la imagen del juego:

Listing 1: Selección de la imagen de memoria en game_top.v

```
pochoco_soc #(
  .MemFile("sw/game.hex")
) u_pochoco (
  .i_Clk(i_Clk),
  // conexiones de LED, botones, displays y SPI
);
```

De esta forma, `game_top` es el top-level sintetizado, mientras que `pochoco_soc.v` mantiene la interconexión entre CPU, RAM y periféricos. No fue necesario modificar el núcleo `espino_core`; la extensión de hardware se realizó en el bloque de periféricos.

3.2. Contador de ciclos incorporado al hardware

Para medir intervalos se agregó en `pochoco_periph.v` el registro `cycle_q`, un contador libre de 32 bits. Después del reinicio global comienza en cero y aumenta en uno en cada flanco positivo del reloj. La Go Board opera a 25 MHz, por lo que el contador avanza 25.000.000 veces por segundo.

Listing 2: Implementación y mapeo del contador de ciclos

```
reg [31:0] cycle_q;

always @(posedge clk_i or negedge rst_ni) begin
  if (!rst_ni) cycle_q <= 32'b0;
  else         cycle_q <= cycle_q + 32'd1;
end

always @(posedge clk_i or negedge rst_ni) begin
  if (!rst_ni) rdata_o <= 32'b0;
  else if (access & ~we_i) begin
    case (off)
      6'd2: rdata_o <= {28'b0, btn_i};
      6'd3: rdata_o <= cycle_q; // desplazamiento 0x0C
      default: rdata_o <= 32'b0;
    endcase
  end
end
```

El índice `off=3` corresponde a $3 \times 4 = 12 =$ `0x0C` bytes desde la base de los periféricos. El contador no se reinicia en cada ronda: el programa guarda una lectura inicial y otra final, y la resta entre ambas entrega el intervalo transcurrido.

4. Juego en Assembly

4.1. Funcionamiento del juego

Al energizar la placa, el programa deja los LED apagados, muestra `0x88` en los displays y espera a que todos los botones estén sueltos. Luego queda a la espera de una primera pulsación, que sirve

para iniciar la partida y, al mismo tiempo, para capturar el valor del contador de ciclos como semilla pseudoaleatoria. Una vez que el jugador suelta el botón, comienza la primera ronda.

Cada ronda parte con los cuatro LED encendidos durante al menos tres segundos. Transcurrido ese tiempo, el sistema elige de forma pseudoaleatoria uno de los cuatro LED como objetivo, lo deja encendido y comienza a medir el tiempo de reacción. El jugador debe presionar lo antes posible el botón que corresponde a ese LED. Si acierta, el tiempo medido se convierte a décimas de segundo, se muestra en los displays y se suma al acumulado de la partida. Si presiona un botón distinto, los displays muestran `EE`, se apagan los LED y el juego aplica una penalización de un segundo. Después reinicia únicamente la ronda actual, por lo que los aciertos y el tiempo acumulado hasta ese momento no se modifican.

La partida termina al completar diez respuestas correctas. El tiempo de la décima ronda se mantiene en los displays hasta que el jugador libere el botón y realice una nueva pulsación; solo entonces el programa calcula el promedio de los diez tiempos, también en décimas de segundo, y lo deja fijo en los displays. Desde ese punto el programa queda en un bucle final y solo se reinicia al resetear la placa. La Figura 2 resume este comportamiento como un diagrama de estados.

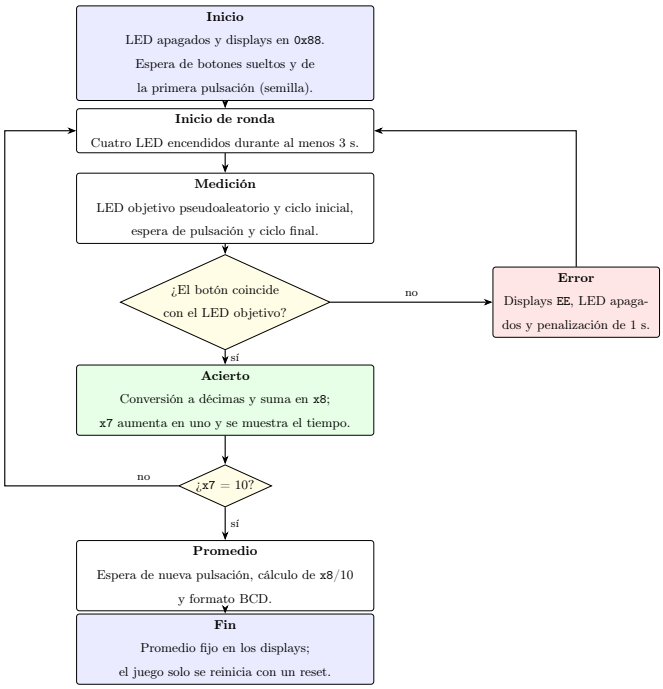


Figura 2: Diagrama de estados del juego.

Fuente: Elaboración propia.

4.2. Registros utilizados

Los registros persistentes principales son `x3` para displays, `x4` para LED, `x5` para botones, `x6` para el contador, `x7` para aciertos correctos, `x8` para la suma de tiempos y `x9` para la semilla. Los registros `x10` a `x15` se reutilizan como temporales.

4.3. Temporización de tres segundos

Al comenzar cada ronda se escribe 15, equivalente a `1111` en binario, en el periférico de LED. Luego se guarda el ciclo inicial y se espera hasta que la diferencia alcance 75.000.000 ciclos. La conversión es:

$$25{,}000{,}000 \ \frac{\text{ciclos}}{\text{s}} \times 3 \text{ s} = 75{,}000{,}000 \text{ ciclos}.$$

Listing 3: Espera de al menos tres segundos

```
addi x10, x0, 15
sw   x10, 0(x4)
```

```
lw   x10, 0(x6)          # ciclo inicial
lui  x11, 0x04787
addi x11, x11, -1856     # 75.000.000 ciclos
wait_three_seconds:
lw   x12, 0(x6)
sub  x13, x12, x10
bltu x13, x11, wait_three_seconds
```

El Listing 3 enciende los cuatro LED escribiendo 15 en la dirección almacenada en `x4`. Luego lee el contador de ciclos a través de `x6` y guarda esa marca de tiempo en `x10` como ciclo inicial.

Dado que la constante de 75.000.000 excede el límite de 12 bits para un operando inmediato, el programa la construye en dos pasos sobre el registro x11, donde primero carga los 20 bits superiores con lui (0x04787) y luego ajusta el valor con addi (-1856), formando la constante hexadecimal 0x047868C0.

Posteriormente, el flujo entra en un bucle de comprobación donde continuamente calcula la diferencia entre el ciclo actual y el inicial (sub). La instrucción de salto condicional bltu evalúa esta diferencia en cada iteración y mantiene a la CPU en este ciclo de forma activa, y solo cuando el delta de ciclos alcanza o supera el equivalente a tres segundos, la condición deja de cumplirse y el programa avanza a la siguiente etapa del juego.

4.4. Generación pseudoaleatoria del LED objetivo

La secuencia no comienza siempre con una semilla fija, en su lugar, el programa implementa un bucle de espera sobre el periférico de los botones (x5). Una vez detectada la primera pulsación, se lee el contador de ciclos (x6), y se guarda en el registro x9 como semilla base. A continuación se detalla la implementación de lo descrito:

Listing 4: Captura de la semilla inicial

```
wait_start_press:
lw x10, 0(x5)
beq x10, x0, wait_start_press
lw x9, 0(x6)  # Se captura el contador de ciclos como semilla
```

En cada ronda se mezcla esa semilla con la última lectura del contador (`x12`), la cantidad de aciertos acumulados y una unidad. Como los tiempos de pulsación y respuesta varían entre partidas, también varía la secuencia obtenida.

Listing 5: Generación del índice pseudoaleatorio

```
add  x9, x9, x12
add  x9, x9, x7
addi x9, x9, 1
andi x10, x9, 3          # indice 0, 1, 2 o 3
```

La operación `andi` con 3 conserva los dos bits menos significativos y produce un índice entre 0 y 3. Después, una serie de branches convierte ese índice en uno de los patrones `0001`, `0010`, `0100` o `1000`. Se usan branches porque las instrucciones de desplazamiento están deshabilitadas en la ALU de Espino:

Listing 6: Conversión del índice al patrón del LED

```
beq x10, x0, target_led_1
addi x11, x0, 1
beq x10, x11, target_led_2
addi x11, x0, 2
beq x10, x11, target_led_4
addi x11, x0, 8
jal x0, show_target
```

4.5. Medición y conversión a décimas de segundo

Después de escribir el LED objetivo, el programa lee inmediatamente el contador y guarda esa lectura en `x12` como ciclo inicial. Cuando detecta una pulsación, toma una segunda lectura y la guarda en `x14` como ciclo final. La diferencia `x14 - x12` corresponde al tiempo de reacción en ciclos. Una décima de segundo equivale a:

$$25{,}000{,}000 \ \frac{\text{ciclos}}{\text{s}} \times 0{,}1 \text{ s} = 2{,}500{,}000 \text{ ciclos.}$$

El programa resta 2.500.000 repetidamente y aumenta un contador por cada resta completa. Así obtiene el tiempo en décimas sin necesitar una instrucción de división. Las fracciones menores a una décima se truncan.

Listing 7: Conversión de ciclos a décimas de segundo

```
sub  x15, x14, x12       # ciclos de respuesta
lui  x10, 0x00262
addi x10, x10, 1440      # 2.500.000 ciclos
addi x12, x0, 0          # resultado en decimas
cycles_to_tenths:
bltu x15, x10, time_converted
sub  x15, x15, x10
addi x12, x12, 1
jal  x0, cycles_to_tenths
```

Tal y como se detalla en el listing 7, la constante divisora de 2.500.000 (construida de manera sintética en x10 mediante lui y addi debido al límite de los 12 bits) es restada iterativamente del tiempo total de reacción (x15). En dicho proceso iterativo (cycles_to_tenths) el registro x15 actúa como el dividendo al que se le va descontando el valor, mientras que x12 funciona como el cociente, incrementandose en uno por cada resta completa. La instrucción bltu actúa como condición de salida, esto es, si el residuo de ciclos es estrictamente menor a 2.500.000, el bucle termina. Dicho mecanismo asegura que las fracciones de tiempo inferiores a una décima de segundo sean automáticamente truncadas, entregando un número entero exacto para ser procesado por los displays. El registro `x12` se reutiliza como cociente porque, tras la resta inicial, el ciclo inicial ya no es necesario.

4.6. Validación, errores y promedio

La validación compara el patrón físico de los botones presionado con el patrón del LED objetivo mediante una instrucción de bifurcación (bne). Si son distintos, el flujo deriva a una ruta de error: el programa escribe el valor decimal 238 (0xEE) en los periféricos para mostrar el mensaje "EE" en los displays, apaga los LEDs y ejecuta un bucle de penalización de un segundo (iterando hasta alcanzar 25.000.000 de ciclos). Finalizado este bloqueo temporal, se reinicia únicamente la ronda actual. Dado que esta ruta de error salta las instrucciones de suma, los registros acumuladores (x7

de aciertos y x8 de tiempo) no se modifican, conservando intacto el progreso del jugador.

Listing 8: Manejo de respuesta incorrecta y castigo de 1 segundo

```
wrong_button:
addi x10, x0, 238        # 0xEE (Muestra "EE" en ambos displays)
sw x10, 0(x3)
sw x0, 0(x4)             # Apaga los LEDs

wait_wrong_release:
lw x10, 0(x5)
bne x10, x0, wait_wrong_release

lw x10, 0(x6)            # Captura inicio del castigo
lui x11, 0x017D8         # Construye 25.000.000 ciclos (1 seg)
addi x11, x11, -1984

wait_error_second:
lw x12, 0(x6)
sub x13, x12, x10
bltu x13, x11, wait_error_second
jal x0, round_start      # Vuelve a la ronda sin alterar x7 ni x8
```

Por el contrario, si la respuesta es correcta, el tiempo convertido se suma en x8, x7 aumenta en uno y el tiempo individual se envía a los displays. Al registrar el décimo acierto (x7 = 10), el programa implementa un bloqueo lógico donde exige que el usuario libere el botón y realice una nueva pulsación. Esta espera es relevante, debido a que evita que el tiempo de la décima ronda desaparezca inmediatamente para dar paso al promedio.

Listing 9: Bloqueo lógico tras alcanzar 10 aciertos

```
addi x10, x0, 10
beq x7, x10, wait_tenth_release
# ... (codigo normal de ronda) ...

wait_tenth_release:
```

```
sw x0, 0(x4)             # Apaga el LED objetivo
lw x10, 0(x5)
bne x10, x0, wait_tenth_release

wait_average_press:      # Exige nueva pulsacion deliberada para avanzar
lw x10, 0(x5)
beq x10, x0, wait_average_press

wait_average_release:    # Exige soltar
lw x10, 0(x5)
bne x10, x0, wait_average_release
jal x0, calculate_average
```

Una vez superada esta espera, se procede a calcular el promedio. Como x8 contiene la suma de los diez tiempos expresados en décimas, el programa debe dividir este valor por diez y de manera análoga a la conversión de ciclos, la división se ejecuta de forma sintética mediante restas sucesivas para sortear las limitaciones de la ALU. El registro x15 recibe la suma total y se le resta 10 iterativamente dentro del average_loop, mientras que x12 contabiliza el cociente entero.

Listing 10: Cálculo del promedio de diez respuestas correctas

```
add x15, x8, x0
addi x12, x0, 0
addi x10, x0, 10
average_loop:
bltu x15, x10, average_ready
addi x15, x15, -10
addi x12, x12, 1
jal x0, average_loop
```

Este cociente se formatea a BCD mediante la subrutina display_tenths y se mantiene estático en los displays, mientras el juego entra en un bucle infinito que solo se abandona reiniciando la placa. Como Espino no implementa desplazamientos de bits (shifts), el empaquetado de las decenas hacia el nibble superior se logra multiplicando por 16 mediante cuatro duplicaciones por suma (add) sucesivas.

5. Assembler Desarrollado

El assembler se implementó en Python y recibe dos rutas: el archivo fuente `game.s` y el archivo de salida `game.hex`. La traducción se realiza directamente en el programa y no invoca un assembler RISC-V externo. Su resultado contiene una palabra de 32 bits, escrita con ocho dígitos hexadecimales, por cada instrucción.

5.1. Sintaxis aceptada e instrucciones utilizadas

La sintaxis acepta registros `x0` a `x15`, operandos separados por comas, inmediatos en decimal o con prefijo hexadecimal `0x`, comentarios iniciados por `#` o `//`, etiquetas con la forma `nombre:` y operandos de memoria escritos como `inmediato(registro)`, por ejemplo `0(x5)`. Las directivas que comienzan con punto, como `.section` y `.global`, se ignoran porque no generan instrucciones.

Las instrucciones utilizadas por `game.s` son `lui`, `addi`, `add`, `sub`, `andi`, `lw`, `sw`, `beq`, `bne`, `bltu`, `jal` y `jalr`. El assembler también reconoce otras operaciones del mismo subconjunto, pero el juego no depende de ellas.

5.2. Dos pasadas, etiquetas e inmediatos

El proceso de traducción del código fuente requiere de dos pasadas para poder resolver correctamente las direcciones de memoria. En la primera pasada se limpian los comentarios, se ignoran las directivas y se le asigna una dirección a cada etiqueta encontrada. Este contador de direcciones aumenta en cuatro bytes por cada instrucción, dado que el procesador opera con palabras fijas de 32 bits. Luego, en la segunda pasada, se codifican las instrucciones definitivas, y como las posiciones de todas las etiquetas ya son conocidas, las instrucciones de salto (branch y jal) pueden transformar su destino en un desplazamiento relativo, calculando simplemente la diferencia entre la dirección de la etiqueta y la dirección actual.

Para generar el código máquina, las funciones de codificación se encargan de ubicar el opcode, funct3, funct7, los registros y los inmediatos en las posiciones exactas que define el estándar RISC-V para los formatos R, I, S, B, U y J. Aquí el ensamblador aplica varias validaciones físicas de la arquitectura: los formatos I y S limitan sus inmediatos con signo a 12 bits, mientras que el formato U utiliza 20 bits. Por su parte, los desplazamientos relativos de los branches y de jal deben ser siempre números

pares (debido a la alineación en memoria) y representables en 13 y 21 bits, respectivamente.

El programa incorpora un sistema de validación para evitar cargar un binario defectuoso en la FPGA. El script es capaz de detectar si se usan registros fuera del subconjunto permitido (x0 a x15), instrucciones desconocidas, cantidades incorrectas de operandos o etiquetas repetidas y destinos inexistentes. También bloquea el proceso si encuentra desplazamientos impares o inmediatos fuera de rango. Cuando ocurre alguno de estos errores, la ejecución se detiene y se imprime un mensaje que indica el número exacto y el texto de la línea conflictiva para facilitar la corrección.

5.3. Ejemplo de traducción a código máquina

Para la instrucción `addi x10, x0, 15`, el formato I se distribuye como sigue:

imm[11:0]	rs1	funct3	rd	opcode
000000001111	00000	000	01010	0010011

La concatenación correcta es:

```
00000000111100000000010100010011
```

Agrupada en nibbles queda:

```
0000 0000 1111 0000 0000 0101 0001 0011
```

Por tanto, la palabra escrita en `game.hex` es `00f00513`.

6. Pruebas y Resultados

El assembler se verificó mediante siete pruebas automáticas. Cuatro comparan su salida con los archivos hexadecimales de `7seg`, `blink`, `buttons_leds` y `game`. Las otras tres comprueban que se rechacen un registro inexistente en RV32E, una instrucción desconocida y una etiqueta no definida. Las siete pruebas finalizaron correctamente y `game.hex` regenerado coincidió exactamente con el archivo utilizado por el diseño.

La síntesis de `game_top` terminó sin problemas, la etapa de ubicación y ruteo terminó sin errores y la frecuencia máxima obtenida fue superior a los 25 MHz requeridos. En la Go Board se verificaron el

estado inicial, los cuatro LED encendidos, la selección de objetivos, la detección de botones correctos e incorrectos, la visualización de cada tiempo y la finalización después de diez respuestas correctas. El décimo tiempo se mantiene hasta una nueva pulsación; después de ella se presenta el promedio final.

```
/Library/Developer/CommandLineTools/usr/bin/make -C sw test
python3 -m unittest -v test_assembler.py
test_7seg (test_assembler.ExampleProgramsTest) ... ok
test_blink (test_assembler.ExampleProgramsTest) ... ok
test_buttons_leds (test_assembler.ExampleProgramsTest) ... ok
test_game (test_assembler.ExampleProgramsTest) ... ok
test_rejects_registers_missing_from_rv32e (test_assembler.ValidationTest) ... ok
test_rejects_unknown_instruction (test_assembler.ValidationTest) ... ok
test_rejects_unknown_label (test_assembler.ValidationTest) ... ok

----------------------------------------------------------------------
Ran 7 tests in 0.004s

OK
```

Figura 3: Evidencias de validación del assembler

Fuente: Elaboración propia.

7. Conclusiones

El proyecto permitió integrar software y hardware en un flujo completo: el juego, escrito en assembly RV32E, es traducido por un assembler propio a código máquina, almacenado en la memoria de Pochoco SoC y ejecutado por Espino, cuyas lecturas y escrituras sobre periféricos mapeados en memoria se transforman en acciones físicas sobre la Go Board. De esta forma se comprendió de manera práctica todo el recorrido de un programa, desde `game.s` hasta los LED y displays de la FPGA.

Una de las decisiones centrales fue agregar el contador de ciclos `cycle_q` al bloque de periféricos, lo que permitió implementar la espera de tres segundos, la penalización de un segundo, la semilla pseudoaleatoria y la medición de los tiempos de reacción sin modificar el núcleo ni el conjunto de instrucciones de Espino. Como el procesador no cuenta con instrucciones de división ni de desplazamiento, la conversión de ciclos a décimas de segundo, el cálculo del promedio y el empaquetado en BCD se resolvieron mediante restas y sumas sucesivas. Además, el banco reducido de 16 registros de RV32E obligó a definir una convención clara entre registros persistentes y temporales.

El assembler desarrollado en Python, con lectura en dos pasadas, resolvió las referencias a etiquetas y calculó los desplazamientos relativos de los saltos. También codificó correctamente los formatos R, I, S, B, U y J y validó rangos e inmediatos, lo que garantizó que la memoria recibiera exactamente el código máquina esperado. Las siete pruebas automáticas, la síntesis sin problemas, el ruteo con una frecuencia máxima superior a los 25 MHz requeridos y la verificación en la Go Board respaldan que todas las partes funcionan de manera coordinada: el juego conserva el progreso ante un error, acumula diez tiempos y presenta su promedio en décimas de segundo.

En conclusión, el proyecto cumplió los requisitos de arquitectura, temporización, generación pseudoaleatoria, medición, assembler propio y funcionamiento sobre FPGA. Además, mostró que el desempeño de un sistema no depende solo de su diseño físico, sino también de cómo el software se adapta a los recursos limitados del hardware y de que la traducción entre ambos sea correcta.